

Лабораторная работа № 8.

Развёртывание Node.js REST API на Linux-сервере

Тема: Развёртывание Express API на Ubuntu: Node.js, PostgreSQL, Prisma, systemd и Nginx

Цель: Развернуть Internet Shop REST API на Ubuntu Server, настроить PostgreSQL, systemd, Nginx и сетевой доступ к API.

Дисциплина: Разработка браузерных приложений

Лабораторная работа №8. Развёртывание Node.js REST API на Linux-сервере

В лабораторной работе необходимо развернуть Internet Shop REST API на Ubuntu Server. Node.js-приложение будет запущено как systemd service, PostgreSQL будет работать локально, Nginx будет принимать внешние HTTP-запросы и передавать их Express. Внешний доступ к внутренним портам приложения и базы данных будет закрыт.

1. Ожидаемый результат

- Ubuntu Server доступен по SSH.
- Установлены Node.js 22, PostgreSQL, Git и Nginx.
- Создан отдельный пользователь shop.
- Internet Shop размещён в /var/www/internet-shop.
- Создана production database shop_db.
- Production variables находятся вне Git repository.
- Prisma Client сгенерирован, migrations применены.
- Internet Shop работает как systemd service.
- Nginx проксирует public HTTP requests в Express.
- UFW разрешает SSH и Nginx, но не открывает 3000 и 5432.
- После reboot приложение запускается автоматически.

2. Требования к серверу

- Ubuntu Server 26.04 LTS, 24.04 LTS или 22.04 LTS.
- Доступ в интернет.
- Публичный IP или локальная учебная сеть с маршрутизацией до сервера.
- Пользователь с sudo privileges.
- GitHub repository Internet Shop API.

3. Подключение по SSH

Подключение с Windows 11

```
ssh ubuntu@SERVER_IP
```

Замените SERVER_IP на адрес вашего сервера. После входа все последующие команды выполняются на Ubuntu Server.

4. Обновление Ubuntu

Обновление packages

```
sudo apt update
sudo apt upgrade -y
```

5. Создание пользователя приложения

Пользователь shop

```
sudo adduser shop
sudo usermod -aG sudo shop
```

Node.js REST API не должен постоянно работать от root. В учебной работе владельцем файлов и процесса будет пользователь shop.

6. Установка базовых packages

Git, Nginx и PostgreSQL

```
sudo apt install -y git curl ca-certificates nginx postgresql postgresql-contrib
```

Проверка services

```
sudo systemctl status nginx
sudo systemctl status postgresql
```

7. Установка Node.js 22

Подключение NodeSource repository

```
curl -fsSL https://deb.nodesource.com/setup_22.x -o nodesource_setup.sh
sudo -E bash nodesource_setup.sh
sudo apt install -y nodejs
```

Проверка Node.js

```
node --version
npm --version
which node
```

Для systemd запомните результат which node. В типичной системной установке путь будет /usr/bin/node.

8. Создание каталога приложения

Каталог Internet Shop

```
sudo mkdir -p /var/www/internet-shop
sudo chown -R shop:shop /var/www/internet-shop
```

9. Клонирование repository

Работа от пользователя shop

```
sudo -iu shop
cd /var/www/internet-shop
git clone https://github.com/USER/internet-shop-api.git .
```

Замените URL на фактический repository проекта. Не копируйте node_modules с Windows.

10. Установка dependencies

```
npm ci

cd /var/www/internet-shop
npm ci
```

11. Создание PostgreSQL database

Вход в psql

```
sudo -u postgres psql
```

Пользователь и база

```
CREATE USER shop_user WITH PASSWORD 'CHANGE_ME_STRONG_PASSWORD';
CREATE DATABASE shop_db OWNER shop_user;
\q
```

Используйте собственный сложный пароль. Не публикуйте его в GitHub или отчёте.

12. Создание production environment file

Создание файла

```
sudo nano /etc/internet-shop.env
```

Пример содержимого

```
NODE_ENV=production
PORT=3000
DATABASE_URL=postgresql://shop_user:STRONG_PASSWORD@127.0.0.1:5432/shop_db?schema=public
JWT_SECRET=CHANGE_ME_TO_RANDOM_SECRET
CORS_ORIGIN=https://example.com
```

Если frontend ещё не опубликован, значение CORS_ORIGIN настройте в соответствии с текущей реализацией проекта. Production secrets не должны попадать в repository.

13. Permissions environment file

Ограничение доступа

```
sudo chown root:shop /etc/internet-shop.env
sudo chmod 640 /etc/internet-shop.env
```

14. Временная загрузка variables для Prisma

Чтобы вручную выполнить prisma generate и migrate deploy, экспортируйте необходимые variables в текущую shell-session, не добавляя их в repository.

Загрузка values из файла

```
set -a
source /etc/internet-shop.env
set +a
```

В production automation migrations лучше выполнять отдельным deployment-процессом. В учебной работе мы выполняем команду непосредственно на сервере.

15. Prisma Client и production migrations

Подготовка Prisma

```
cd /var/www/internet-shop
npx prisma generate
npx prisma migrate deploy
```

16. Проверка ручного запуска

Запуск Node.js

```
node server.js
```

В отдельной SSH-session проверьте health endpoints через localhost. После проверки остановите ручной процесс Ctrl+C.

Проверка

```
curl http://127.0.0.1:3000/health/live
curl http://127.0.0.1:3000/health/ready
```

17. Создание systemd service

Unit file

```
sudo nano /etc/systemd/system/internet-shop.service
```

internet-shop.service

```
[Unit]
Description=Internet Shop REST API
After=network.target postgresql.service
```

```
[Service]
Type=simple
User=shop
Group=shop
WorkingDirectory=/var/www/internet-shop
EnvironmentFile=/etc/internet-shop.env
ExecStart=/usr/bin/node /var/www/internet-shop/server.js
Restart=on-failure
RestartSec=5

[Install]
WantedBy=multi-user.target
```

Если `which node` показал другой путь, замените `/usr/bin/node` фактическим абсолютным путём.

18. Запуск systemd service

Активация service

```
sudo systemctl daemon-reload
sudo systemctl enable --now internet-shop
sudo systemctl status internet-shop
```

Сервис должен иметь состояние active (running).

19. Просмотр application logs

```
journalctl

sudo journalctl -u internet-shop -n 100 --no-pager
sudo journalctl -u internet-shop -f
```

20. Проверка API до Nginx

Localhost

```
curl http://127.0.0.1:3000/health/live
curl http://127.0.0.1:3000/api/products
```

Если `localhost:3000` не работает, Nginx пока не настраивайте. Сначала исправьте Node.js application или database connection.

21. Настройка Nginx reverse proxy

Создание сайта

```
sudo nano /etc/nginx/sites-available/internet-shop
```

Nginx configuration

```
server {
    listen 80;
```

```

server_name _;

location / {
    proxy_pass http://127.0.0.1:3000;
    proxy_http_version 1.1;
    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    proxy_set_header X-Forwarded-Proto $scheme;
}
}

```

server_name _ используется для учебной проверки по IP. Если есть domain, замените его на реальное имя, например api.example.com.

22. Включение Nginx site

Проверка и reload

```

sudo ln -s /etc/nginx/sites-available/internet-shop /etc/nginx/sites-enabled/internet-shop
sudo nginx -t
sudo systemctl reload nginx

```

При конфликте с default site его можно отключить, удалив symlink /etc/nginx/sites-enabled/default, затем снова выполнить nginx -t и reload.

23. Проверка через public IP

C Windows 11

```

Invoke-RestMethod -Method Get -Uri "http://SERVER_IP/health/live"
Invoke-RestMethod -Method Get -Uri "http://SERVER_IP/api/products"

```

24. Настройка UFW

Firewall rules

```

sudo ufw allow OpenSSH
sudo ufw allow 'Nginx Full'
sudo ufw enable
sudo ufw status

```

Не выполняйте ufw enable до разрешения OpenSSH, иначе можно потерять удалённый доступ к VPS.

25. Проверка закрытых внутренних портов

Express должен обслуживаться через Nginx. Не создавайте UFW rules для 3000/tcp и 5432/tcp.

Слушающие sockets

```

sudo ss -lntp | grep -E ':22|:80|:443|:3000|:5432'

```

26. Настройка HTTPS — при наличии domain

Если у вас есть domain и DNS A-record уже указывает на SERVER_IP, замените server_name _ на domain и установите certificate. Этот этап можно пропустить в лабораторной работе, если domain отсутствует.

```
Certbot

sudo snap install --classic certbot
sudo ln -s /snap/bin/certbot /usr/local/bin/certbot
sudo certbot --nginx -d api.example.com
```

27. Проверка HTTPS

```
Публичный HTTPS endpoint

Invoke-WebRequest -Method Get -Uri "https://api.example.com/health/ready"
```

28. Проверка автозапуска

Перезагрузите учебный сервер только если это разрешено преподавателем или средой лаборатории.

```
Reboot

sudo reboot
```

После повторного SSH-подключения проверьте `systemctl status internet-shop` и public API. Service должен запуститься автоматически.

29. Диагностика 502 Bad Gateway

1. Проверьте `sudo systemctl status internet-shop`.
2. Откройте `sudo journalctl -u internet-shop -n 100`.
3. Проверьте `curl http://127.0.0.1:3000/health/live`.
4. Проверьте `sudo nginx -t`.
5. Откройте `/var/log/nginx/error.log`.
6. После исправления перезапустите нужный service.

30. Полезные команды обслуживания

Команды

Задача	Команда
Status API	<code>sudo systemctl status internet-shop</code>
Restart API	<code>sudo systemctl restart internet-shop</code>
API logs	<code>sudo journalctl -u internet-shop -f</code>
Status PostgreSQL	<code>sudo systemctl status postgresql</code>
Test Nginx config	<code>sudo nginx -t</code>

Задача	Команда
Reload Nginx	<code>sudo systemctl reload nginx</code>
Nginx error log	<code>sudo tail -f /var/log/nginx/error.log</code>
Firewall	<code>sudo ufw status</code>

31. Контрольные вопросы

- Почему Node.js REST API запускается не от root?
- Зачем нужен systemd?
- Что делает Restart=on-failure?
- Почему DATABASE_URL использует 127.0.0.1?
- Для чего нужен prisma migrate deploy?
- Почему порт 3000 не открывается в UFW?
- Какую роль выполняет Nginx?
- Что означает 502 Bad Gateway?
- Где находятся application logs Node.js service?
- Почему сначала проверяют localhost:3000, а потом public URL?
- Что нужно для настройки HTTPS?

32. Требования к отчёту

- Цель лабораторной работы.
- Версия Ubuntu и Node.js.
- Скриншот `systemctl status postgresql`.
- Скриншот структуры `/var/www/internet-shop` без secrets.
- Фрагмент systemd unit file.
- Скриншот `systemctl status internet-shop`.
- Фрагмент journalctl с успешным startup.
- Nginx configuration.
- Результат `nginx -t`.
- Скриншот `ufw status`.
- Результат `/health/live`.
- Результат `/health/ready`.
- Результат GET `/api/products` через public IP или domain.
- Если настроен HTTPS — результат HTTPS-запроса.
- Ответы на контрольные вопросы.

33. Критерии выполнения

Критерии

Критерий	Результат
Node.js 22 установлен	Обязательно
PostgreSQL работает локально	Обязательно
Production DB и user созданы	Обязательно
Prisma Client и migrations подготовлены	Обязательно
systemd service active	Обязательно

Критерий	Результат
Nginx reverse проху работает	Обязательно
Port 3000 не открыт через UFW	Обязательно
Public REST endpoint отвечает	Обязательно
Service включён в автозапуск	Обязательно
HTTPS	Дополнительно при наличии domain

34. Вывод

В результате лабораторной работы Internet Shop REST API развёрнут на Ubuntu Server без PaaS. Node.js process управляется systemd, PostgreSQL работает локально, Prisma поддерживает структуру database, Nginx публикует REST API наружу, а UFW ограничивает внешний доступ только необходимыми сервисами.